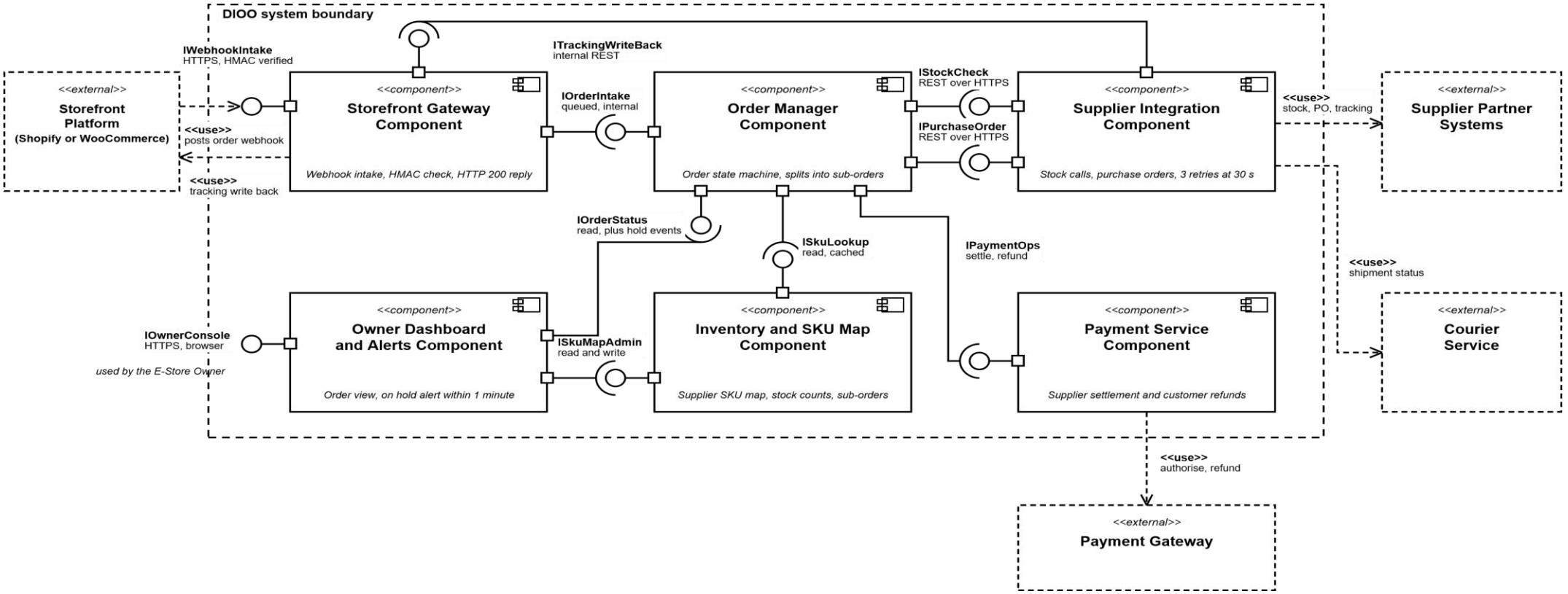
Dropshipping Inventory and Order Orchestrator (DIOO): UML component diagram

Microservices architecture. Akash Singh, PES2UG24CS042. SE Lab 3



Notation

- Provided interface (ball): a service the component offers. A ball on the boundary with no socket is a public port used from outside the system.
- Required interface (socket): a service the component needs. A ball sitting inside a socket is an assembly connector between the two components.
- Port: the interaction point on the component boundary. Each connector is labelled with the interface name and the transport used.
- Usage dependency <<use>>: a call to a system outside the DIOO boundary. Dashed boxes are external systems, not DIOO components.

Actors from Lab 1: Storefront Platform posts the order webhook, Supplier Partner returns stock and tracking, Courier Service reports shipment status, E-Store Owner works from the dashboard.

Six components inside the boundary, eight interfaces between components, two public ports, five usage dependencies on external systems.

PES University
Software Engineering Laboratory
Lab 3: Component modelling and architectural pattern selection

Architecture selection for the dropshipping inventory and order orchestrator

Why the system is built as microservices rather than as a layered or client server application

Name: Akash Singh
SRN: PES2UG24CS042
Lab: Lab 3, component modelling and architectural pattern selection
Project: Dropshipping Inventory and Order Orchestrator, problem statement 38
Actors: E-Store Owner, Supplier Partner, Storefront Platform, Courier Service

1. Architecture selection

We chose the microservices architecture for the Dropshipping Inventory and Order Orchestrator system. It is split into six components inside one boundary: Storefront Gateway, Order Manager, Supplier Integration, Inventory and SKU Map, Payment Service, and Owner Dashboard and Alerts. Order Manager and Payment Service are the two components the handout fixed. The customer pays on the storefront before the order webhook arrives, so the Payment Service here pays the suppliers and handles the refund the owner issues when a sub-order cannot be filled.

Component	Lab 1 requirements it covers
Storefront Gateway	FR-001 webhook intake, FR-004 tracking write back, NFR-001 signature check and reply time
Order Manager	FR-001 order run, FR-002 split into sub-orders, NFR-002 retry policy and queue
Supplier Integration	FR-001 stock check and purchase order, FR-004 tracking, NFR-002 three retries
Inventory and SKU Map	FR-002 supplier per SKU, UC-08 manage supplier SKU map, stock counts
Payment Service	Refund and supplier settlement on the FR-003 hold path
Owner Dashboard and Alerts	FR-003 on hold alert within one minute, FR-005 order dashboard

2. Styles considered

A layered design would ship the presentation, business and data layers as one unit. The webhook listener, the supplier retry loop and the owner dashboard would scale together, and a change to supplier handling would redeploy all of them. A client server design keeps the same logic in one process, so a supplier API that hangs holds threads the webhook reply needs. Microservices adds network hops and more to run. That cost is worth paying here because these parts of the system run on very different timings and fail in different ways.

3. Two reasons for the choice

1. NFR-001 and NFR-002 ask for opposite things. The webhook has to be answered with HTTP 200 in under 200 ms once the HMAC signature checks out. A failed supplier call has to be retried three times with 30 seconds between attempts, so one order can sit in supplier work for over a minute. With separate services the Storefront Gateway verifies the signature, queues the order and replies, while Supplier Integration runs the slow retry loop. In one process those retries would compete with the reply deadline.
2. One supplier going down should not stop the rest. DIOO holds no stock of its own, so every order depends on supplier APIs, and those go down often. Supplier Integration sits apart from intake and the dashboard, so an outage only puts the affected sub-order On Hold and alerts the owner. Intake, the dashboard and the other suppliers keep working, and nothing leaves the queue, which is what NFR-002 asks for.

4. Security advantage

Only the Storefront Gateway is reachable from the internet. Its one job at that edge is to verify the HMAC signature and reject anything unsigned, so a forged webhook never reaches the Order Manager or the supplier path. Credentials also sit with the component that needs them. Supplier API keys stay in Supplier Integration and payment gateway keys stay in Payment Service, so breaking into the gateway does not give an attacker either of them. One process would hold all of these secrets in the same place.

5. Performance benefit

The reply time comes from keeping slow work off the webhook path. The gateway checks the signature, writes the order to the queue and returns HTTP 200. Stock checks and purchase order dispatch happen after that in Supplier Integration, where a call can take seconds and may be retried. When a burst of orders comes in, only the gateway and the Order Manager need more instances, which is what NFR-001 asks for at 100 webhooks per second.